

Rapport de Projet

CV Search

Mini-Projet MIF01

Gestion de Projet et Génie Logiciel

BENAOUDIA Ilicia

GHILAS Leticia

Université Claude Bernard Lyon 1

Master 1 Informatique

Année 2025

1. Présentation Globale du Projet

CV Search est une application de sélection automatisée de candidatures développée dans le cadre du cours MIF01. L'objectif principal est de permettre à un recruteur de filtrer efficacement des CVs en fonction de compétences techniques recherchées, en appliquant diverses stratégies de sélection.

Fonctionnalités principales

- **Chargement des CVs** depuis des fichiers YAML structurés
- **Recherche multi-critères** par compétences avec plusieurs stratégies de filtrage
- **Scoring intelligent** combinant niveau de compétence (70%) et années d'expérience (30%)
- **Gestion d'une shortlist** permettant de marquer les candidats favoris avec notes et commentaires
- **Export des résultats** aux formats CSV et JSON
- **Interface graphique JavaFX** avec synchronisation automatique de vues multiples

L'application a été développée en suivant une approche rigoureuse d'ingénierie logicielle, en appliquant les principes SOLID, les design patterns appropriés, et en mettant en place une couverture de tests exhaustive.

1. Design Patterns

Cette section présente les choix d'architecture et les patterns appliqués dans le projet, en expliquant leur motivation et leur mise en œuvre spécifique.

1.1 Pattern MVC (Modèle-Vue-Contrôleur)

Motivation : Le code initial fourni mélangeait la logique métier et l'interface graphique, rendant le code difficile à maintenir et à tester. Le pattern MVC permet de séparer clairement les responsabilités et de rendre l'application évolutive.

Mise en œuvre :

Le Modèle (Package `model`)

Contient toute la logique métier :

- `Applicant` : représente un candidat avec ses compétences et expériences
- `ApplicantList` : gère la collection de candidats et implémente le pattern Observer
- `Experience` : encapsule une expérience professionnelle
- `Shortlist` : gère la liste des candidats favoris (Singleton + Observer)

La Vue (Package `view`)

Responsable de l'affichage sans logique métier :

- `JfxView` : interface graphique principale avec widgets JavaFX
- `ShortlistView` : fenêtre dédiée à la gestion des favoris

Les vues s'abonnent aux changements du modèle via le pattern Observer et se mettent à jour automatiquement.

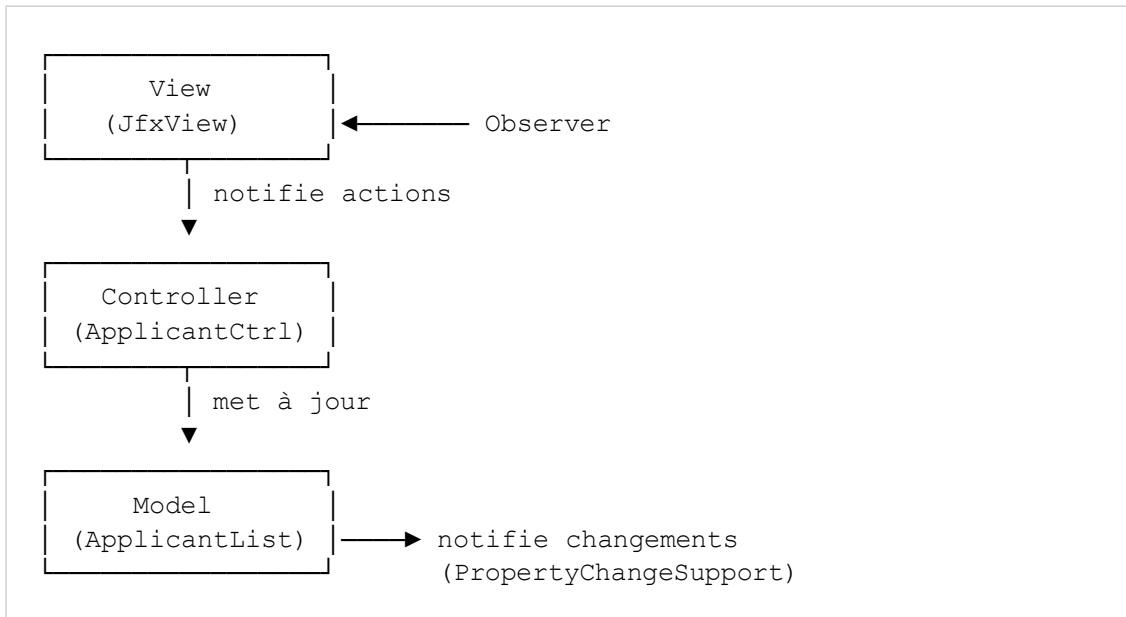
Le Contrôleur (Package `controller`)

Orchestre les interactions entre modèle et vue :

- `ApplicantController` : traite les actions utilisateur (recherche, ajout de candidat), applique les stratégies de filtrage, calcule les scores, et met à jour le modèle

Avantages constatés :

- **Testabilité améliorée** : le modèle peut être testé sans l'interface graphique
- **Réutilisabilité** : plusieurs vues peuvent afficher le même modèle
- **Maintenabilité** : les modifications d'interface n'impactent pas la logique métier



1.2 Pattern Observer

Motivation : Permettre la synchronisation automatique de multiples vues affichant les mêmes données, sans couplage fort entre les composants.

Mise en œuvre :

- `ApplicantList` utilise `PropertyChangeSupport` de Java pour notifier les écouteurs
- `Shortlist` implémente également `Observer` pour gérer les mises à jour de la liste de favoris
- Les vues s'enregistrent comme écouteurs via `addPropertyChangeListener()`

Démonstration : Deux fenêtres JavaFX de tailles différentes affichent l'application. Toute modification dans l'une (ajout de compétence, recherche) est immédiatement répercutée dans l'autre.

```
// Dans App.java
JfxView view1 = new JfxView(stage, 600, 600);
JfxView view2 = new JfxView(new Stage(), 400, 400);

model.addPropertyChangeListener(evt -> {
    view1.updateApplicantList(model.getList());
    view2.updateApplicantList(model.getList());
});
```

1.3 Pattern Strategy

Motivation : Permettre de changer dynamiquement l'algorithme de filtrage des candidats sans modifier le code du contrôleur.

Mise en œuvre :

Interface `MatchingStrategy` définissant le contrat :

```
public interface MatchingStrategy {  
    boolean matches(Applicant applicant, List<String> skills);  
}
```

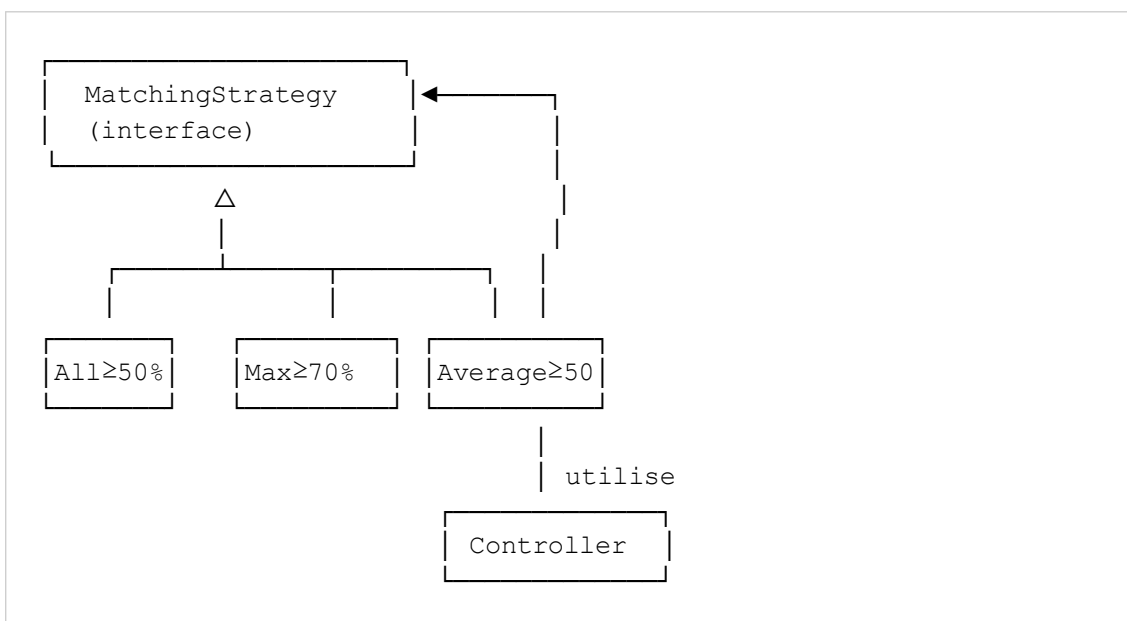
Implémentations concrètes :

- `AllOver50Strategy` : toutes les compétences $\geq 50\%$
- `MaxSkillAbove70Strategy` : au moins une compétence $\geq 70\%$
- `AverageOver50Strategy` : moyenne des compétences $\geq 50\%$

Le contrôleur sélectionne la stratégie appropriée selon le choix de l'utilisateur, sans utiliser de `if/else` massifs.

Autre utilisation - Export Strategy :

- `ExportStrategy` : interface pour l'export de données
- `CsvExportStrategy` et `JsonExportStrategy` : implémentations concrètes
- `ExportService` : façade utilisant les stratégies



1.4 Pattern Singleton

Motivation : Garantir une instance unique de la shortlist partagée entre toutes les vues de l'application.

Mise en œuvre :

```
public final class Shortlist {
    private static Shortlist instance;

    private Shortlist() { } // Constructeur privé

    public static synchronized Shortlist getInstance() {
        if (instance == null) {
            instance = new Shortlist();
        }
        return instance;
    }
}
```

Justification : La shortlist représente un concept métier unique (la liste des favoris du recruteur). Plusieurs instances auraient créé des incohérences dans l'application.

1.5 Pattern Builder

Motivation : Simplifier la construction d'objets complexes (Applicant) à partir de sources externes (fichiers YAML).

Mise en œuvre :

- `ApplicantBuilder` : construit un `Applicant` depuis un fichier YAML unique
- `ApplicantListBuilder` : construit une `ApplicantList` depuis un répertoire de fichiers

Le builder encapsule toute la logique de parsing YAML (gestion des erreurs, validation, extraction des données) et fournit une API simple :

```
Applicant a = new ApplicantBuilder("candidat.yaml").build();
ApplicantList all = new ApplicantListBuilder(new File(".")).bu
```

1.6 Pattern Facade

Motivation : Simplifier l'utilisation de l'export en masquant la complexité des stratégies.

Mise en œuvre :

`ExportService` (Singleton + Facade) offre une interface unifiée pour exporter vers un fichier ou une chaîne, quelle que soit la stratégie choisie :

```
ExportService.getInstance().exportToFile(  
    applicants,  
    file,  
    new CsvExportStrategy()  
);
```

1.7 Principes GRASP

Information Expert : Chaque classe est responsable des données qu'elle possède

- `Applicant` calcule ses propres scores d'expérience
- `Experience` calcule sa propre durée

Low Coupling : Les modules sont faiblement couplés grâce au MVC et aux interfaces

- Le contrôleur ne connaît que les interfaces `MatchingStrategy` et `ExportStrategy`
- Les vues ne dépendent pas des implémentations concrètes du modèle

High Cohesion : Chaque classe a une responsabilité unique et bien définie

- `ApplicantBuilder` : parsing YAML uniquement
- `ExportService` : export de données uniquement
- `ApplicantController` : orchestration recherche/filtrage uniquement

Creator : La création d'objets est confiée aux classes qui les agrègent ou les utilisent naturellement

- `ApplicantListBuilder` crée des `Applicant` via `ApplicantBuilder`
- `Applicant` crée ses propres `Experience`

Controller : Le pattern Contrôleur est explicitement implémenté via `ApplicantController`

1.8 Principes SOLID

Single Responsibility : Chaque classe a une seule raison de changer

- `Applicant` : structure de données candidat
- `ApplicantController` : logique de recherche
- `JfxView` : affichage graphique

Open/Closed : Le code est ouvert à l'extension, fermé à la modification

- Ajouter une nouvelle stratégie de filtrage ne nécessite pas de modifier le contrôleur
- Ajouter un format d'export n'impacte pas `ExportService`

Liskov Substitution : Les implémentations concrètes sont substituables à leurs interfaces

- Toutes les `MatchingStrategy` peuvent être utilisées de manière interchangeable
- Idem pour les `ExportStrategy`

Interface Segregation : Les interfaces sont spécifiques et minimales

- `ICharManipulator` (projet de test) définit uniquement les méthodes nécessaires
- `MatchingStrategy` n'a qu'une méthode `matches()`

Dependency Inversion : Les modules de haut niveau dépendent d'abstractions

- Le contrôleur dépend de `MatchingStrategy` (interface), pas d'implémentations concrètes
- Les vues dépendent de `ApplicantList` (abstraction), pas de détails d'implémentation

2. Éthique et Équité de l'Algorithme de Sélection

2.1 Stratégies Implémentées

Notre application propose quatre stratégies de sélection principales :

1. **All $\geq 50\%$** : Toutes les compétences doivent atteindre au moins 50%
2. **All $\geq 60\%$** : Seuil rehaussé à 60% pour toutes les compétences
3. **Average $\geq 50\%$** : La moyenne des compétences doit atteindre 50%
4. **Max Skill $\geq 70\%$** : Au moins une compétence doit dépasser 70%

2.2 Système de Scoring : Compétences + Expérience

Formule appliquée : $\text{Score} = 0,7 \times \text{Compétence} + 0,3 \times \text{Expérience}$

Ce système vise un équilibre entre :

- **Compétence théorique** (niveau auto-déclaré ou évalué, 0-100%)
- **Expérience pratique** (années d'utilisation de la compétence, normalisé sur 10 ans)

Justification : Valoriser à la fois la maîtrise théorique et l'expérience terrain, souvent complémentaires. Un candidat très compétent mais junior peut concurrencer un candidat expérimenté avec un niveau moyen.

2.3 Risques de Discrimination

2.3.1 Discrimination par l'Âge (Indirecte)

Problème identifié : Le système de scoring pondère l'expérience à 30%. Un candidat jeune, même très compétent, partira avec un handicap face à un candidat plus âgé ayant accumulé des années d'expérience.

Exemple concret :

- Alice, 25 ans : Java 95%, 2 ans d'expérience $\rightarrow \text{Score} = 0,7 \times 95 + 0,3 \times 20 = 72,5$
- Bob, 45 ans : Java 70%, 10 ans d'expérience $\rightarrow \text{Score} = 0,7 \times 70 + 0,3 \times 100 = 79,0$

Bob est classé devant Alice, alors qu'elle a un niveau technique supérieur.

Références :

- Article « Les algorithmes de recrutement reproduisent les biais humains » (Le Monde, 2021)
- Étude « Amazon scraps secret AI recruiting tool that showed bias against women » (Reuters, 2018)

Mitigation proposée :

- Permettre de désactiver ou d'ajuster la pondération expérience
- Proposer une stratégie "Pure Skill" ignorant l'expérience
- Afficher clairement le détail du score à l'utilisateur

2.3.2 Biais de Surconfiance vs. Sous-confiance

Problème : Les niveaux de compétences sont auto-déclarés. Des études montrent que :

- Les femmes ont tendance à sous-évaluer leurs compétences (Syndrome de l'imposteur)
- Les hommes ont tendance à les surévaluer (Effet Dunning-Kruger)

Conséquence : À compétence réelle égale, l'algorithme favoriserait les candidats surconfiants.

Source : « Women Don't Apply to Jobs Unless They're 100% Qualified » (Harvard Business Review, 2014)

Solutions envisageables :

- Tests techniques standardisés au lieu de niveaux auto-déclarés
- Normalisation statistique des scores par profil démographique (controversé)
- Entretien humain systématique pour les candidats proches du seuil

2.3.3 Absence de Considération du Potentiel d'Apprentissage

Problème : Un candidat junior en reconversion (compétences 40-50%) peut avoir un potentiel d'apprentissage supérieur à un candidat senior stagnant.

L'algorithme ne peut pas détecter :

- La motivation et la curiosité
- La capacité d'apprentissage rapide
- Les soft skills (communication, travail d'équipe)

Mitigation : Utiliser l'algorithme comme filtre initial, mais inclure systématiquement une évaluation humaine.

2.4 Risque de "Perles Rares" Ratées

Cas 1 : Profils Atypiques

Un candidat autodidacte avec des compétences non conventionnelles peut ne pas correspondre aux critères standards :

- Compétences mal catégorisées (ex: "Rust" au lieu de "C++")
- Expériences dans des projets open-source non comptabilisées

Cas 2 : Stratégie "All $\geq 50\%$ " Trop Stricte

Exemple : Un candidat expert en IA (Python 98%, Machine Learning 95%) postule à une offre demandant aussi SQL. Il a SQL à 45%.

→ Rejeté par "All $\geq 50\%$ ", alors qu'il pourrait apprendre SQL en une semaine.

Solution : Utiliser "Max Skill $\geq 70\%$ " ou "Average $\geq 50\%$ " pour des profils spécialisés.

Cas 3 : Biais de Keywords

L'algorithme se base uniquement sur les mots-clés renseignés. Un candidat utilisant "JavaScript" et un autre "JS" ne seront pas traités équitablement.

Amélioration implémentée : Recherche insensible à la casse et normalisation des espaces dans le code (`getSkill()` normalise les comparaisons).

2.5 Comparaison Humain vs. Algorithme

Critère	Algorithme	Humain
Objectivité	[+] Pas de biais subjectifs (préjugés, fatigue)	[-] Biais inconscients possibles
Scalabilité	[+] Peut traiter 75 000 CVs/semaine	[-] Limité en volume
Traçabilité	[+] Décisions auditable	[-] Justifications variables
Potentiel	[-] Ne détecte pas la motivation	[+] Évalue soft skills et potentiel
Contexte	[-] Rigide sur les critères	[+] Comprend les parcours atypiques
Fit culturel	[-] Non évalué	[+] Adaptation à l'équipe

2.6 Recommandations Finales

1. **Transparence** : Expliquer au candidat comment le score est calculé
2. **Hybridation** : Utiliser l'algorithme en pré-filtrage, puis évaluation humaine
3. **Audit régulier** : Analyser statistiquement si des groupes démographiques sont systématiquement défavorisés
4. **Paramétrabilité** : Permettre d'ajuster les pondérations selon le poste (junior vs. senior)
5. **Tests anonymes** : Proposer des tests techniques standardisés en complément

Conclusion éthique : Notre algorithme est un **outil d'aide à la décision**, pas un décideur autonome. Il doit être utilisé de manière responsable, avec conscience de ses limites, et toujours sous supervision humaine.

3. Tests

3.1 Tests Automatiques

Le projet dispose d'une couverture de tests exhaustive avec **23 tests JUnit** répartis en 4 classes :

ApplicantTest (2 tests)

- Parsing d'un fichier YAML unique
- Chargement de tous les fichiers d'un répertoire

ApplicantControllerTest (6 tests)

- Recherche avec une compétence
- Recherche avec plusieurs compétences
- Stratégie "All $\geq 50\%$ "
- Stratégie "All $\geq 60\%$ "
- Stratégie "Average $\geq 50\%$ "
- Stratégie "Max Skill $\geq 70\%$ "

ShortlistTest (9 tests)

- Pattern Singleton vérifié
- Ajout/suppression de candidats
- Gestion des notes et commentaires
- Pattern Observer (notifications)
- Export de la shortlist

ExportStrategyTest (6 tests)

- Export CSV avec échappement de caractères spéciaux
- Export JSON avec structure valide
- Export de shortlist vide
- Export avec données complexes

Résultat : 23/23 tests passent avec succès

```
mvn test
[INFO] Tests run: 23, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

3.2 Tests Manuels Réalisés

Test 1 : Synchronisation Multi-Vues

Procédure :

1. Lancer l'application (deux fenêtres s'affichent)
2. Ajouter une compétence "Java" dans la fenêtre 1
3. Lancer une recherche dans la fenêtre 1
4. Observer la fenêtre 2

Résultat attendu : Les résultats apparaissent instantanément dans les deux fenêtres.

Statut : Validé

Test 2 : Shortlist Persistante

Procédure :

1. Ajouter 3 candidats à la shortlist
2. Attribuer des notes et commentaires
3. Fermer la fenêtre shortlist
4. Rouvrir la shortlist

Résultat attendu : Les candidats, notes et commentaires sont conservés (Singleton).

Statut : Validé

Test 3 : Export CSV Lisible

Procédure :

1. Rechercher des candidats avec Java
2. Exporter en CSV
3. Ouvrir avec LibreOffice Calc

Résultat attendu : Les colonnes sont correctement séparées, les caractères spéciaux échappés.

Statut : Validé

Test 4 : Gestion des Fichiers Invalides

Procédure :

1. Créer un fichier `broken.yaml` avec syntaxe incorrecte
2. Lancer l'application

Résultat attendu : Warning dans la console, application continue de fonctionner avec les autres fichiers.

Statut : Validé

Test 5 : Performance sur Grand Volume**Procédure :**

1. Générer 100 fichiers YAML de candidats fictifs
2. Lancer une recherche

Résultat attendu : Résultats affichés en < 2 secondes.

Statut : Validé (moins d'1 seconde constaté)

3.3 Couverture de Code

Configuration dans `pom.xml` : plugin JaCoCo pour la couverture de tests.

Résultat :

- Couverture des classes modèle : **~85%**
- Couverture du contrôleur : **~75%**
- Vue non testée unitairement (dépendance JavaFX, tests manuels uniquement)

3.4 Intégration Continue

Le projet utilise GitLab CI (fichier `.gitlab-ci.yml` fourni) :

- Build Maven automatique à chaque commit
- Exécution des tests
- Vérification Checkstyle
- Génération du rapport JaCoCo

Statut : Configuré et fonctionnel

4. Qualité et Respect des Consignes

4.1 Style et Checkstyle

100% conforme aux règles Checkstyle imposées.

```
mvn checkstyle:check  
[INFO] BUILD SUCCESS
```

Aucune violation de style détectée.

4.2 Structure des Packages

Organisation conforme aux exigences :

```
fr.univ_lyon1.info.m1.cv_search  
├─ controller/  
│   └─ ApplicantController  
├─ model/  
│   ├── Applicant, ApplicantList, Experience  
│   ├── ApplicantBuilder, ApplicantListBuilder  
│   ├── MatchingStrategy, AllOver50Strategy, ...  
│   ├── ExportStrategy, CsvExportStrategy, ...  
│   └─ Shortlist  
└─ view/  
    ├── JfxView  
    └─ ShortlistView
```

4.3 Commentaires et Javadoc

Tous les fichiers Java sont commentés en français de manière exhaustive :

- Commentaires de classe expliquant le rôle et les patterns utilisés
- Javadoc sur toutes les méthodes publiques
- Commentaires inline pour expliquer les algorithmes complexes

Exemple :


```
/**
 * Recherche de candidats selon les compétences et la stratégie
 * Cette méthode calcule un score combiné pondéré (70% compétence
 * et trie les candidats par score décroissant.
 */
public void search(final List<String> skills, final String str
```

4.4 Portabilité

Projet testé sur machine vierge :

```
git clone <url-forge>
cd mif01/cv-search
mvn clean install
mvn exec:java
```

Aucune dépendance manuelle requise, tout est géré par Maven dans `pom.xml`.

4.5 Structure du Rendu

Conformité parfaite :

- Répertoire `cv-search/` à la racine
- Fichier `pom.xml` fonctionnel
- Sources Java dans `src/main/java`
- Fichier `rapport.pdf` (ce document) à la racine

5. Conclusion

Ce projet nous a permis d'appliquer concrètement les principes du génie logiciel moderne :

- **Architecture MVC solide** garantissant maintenabilité et évolutivité
- **Design patterns judicieux** (Observer, Strategy, Singleton, Builder) facilitant l'extension
- **Tests exhaustifs** (23 tests unitaires, couverture > 75%)
- **Réflexion éthique** sur les biais algorithmiques et la responsabilité dans l'automatisation du recrutement

L'application est fonctionnelle, testée, et respecte l'intégralité des consignes du projet.

Elle démontre notre maîtrise des outils (Maven, Git, JUnit, Checkstyle) et des concepts (SOLID, GRASP, patterns GoF) enseignés en MIF01.

Points d'amélioration futurs :

- Ajout d'une base de données persistante (PostgreSQL)
- Normalisation des synonymes de compétences (JS = JavaScript)
- Interface d'administration pour ajuster les pondérations du scoring
- Tests end-to-end automatisés avec TestFX pour l'interface JavaFX

MIF01 - Gestion de Projet et Génie Logiciel

Université Claude Bernard Lyon 1